Autonomous Self-Healing and Adaptive Multipath Routing System for Fault-Tolerant Communication Networks

1st Navyansh Tripathi
Department of Electronics and Communication
MIT Manipal
Karnataka,India
navyansh.mitmpl2023@learner.manipal.edu

2nd Shrey Shukla
Department of Electronics and Communication
MIT Manipal
Karnataka,India
shrey.mitmpl2023@learner.manipal.edu

***Abstract*—This project proposes a fault-tolerant communication network model that can automatically detect failures, recover from faults, and intelligently distribute traffic across multiple network paths. The system is designed to simulate real-world network resilience mechanisms using Python, focusing on self-healing, adaptive routing, and multipath traffic management. Unlike conventional static routing systems, the proposed model dynamically monitors network conditions and autonomously performs recovery and optimization actions without manual intervention.**

***Index Terms*—Self-healing networks, Adaptive multipath routing, Fault-tolerant communication, Network resilience**

I. Introduction

Modern communication networks must maintain reliable data transmission despite failures, congestion, and dynamic traffic conditions. Conventional static routing mechanisms lack the ability to adapt to such changes, leading to reduced performance and network instability. This project presents an autonomous self-healing and adaptive multipath routing system that enables fault-tolerant communication by automatically detecting network failures, dynamically reconfiguring routes, and intelligently distributing traffic across multiple paths. The proposed system improves network resilience, reliability, and overall communication efficiency.

II. Core Features

A. Autonomous Self-Healing Mechanism

The self-healing mechanism enables the network to automatically detect and recover from failures without manual intervention. The system continuously monitors network links and nodes to identify abnormal conditions such as link breakdowns, node crashes, and congestion overload. When a failure is detected, the network control module triggers an automatic recovery process that recalculates alternative paths and reroutes traffic in real time. This ensures uninterrupted communication and minimizes packet loss, delay, and service disruption. The self-healing process allows the network to maintain stability and reliability even in dynamic and failure-prone environments.

Identify applicable funding agency here. If none, delete this.

III. Adaptive Routing System

The adaptive routing system dynamically selects optimal communication paths based on real-time network conditions. Instead of relying on static routing tables, the system evaluates multiple performance parameters such as latency, bandwidth availability, congestion level, packet loss, and link reliability. Based on these metrics, routing decisions are continuously updated to ensure efficient data transmission. This adaptive behavior allows the network to respond intelligently to changing traffic patterns and network states, resulting in improved performance, better load distribution, and enhanced quality of service.

A. Adaptive Multipath Traffic Splitting (AMTS)

Adaptive Multipath Traffic Splitting is a key distinguishing feature of the project. The system identifies multiple viable communication paths between source and destination nodes and distributes traffic across these paths instead of using a single route. Traffic allocation is performed dynamically based on path quality metrics such as available bandwidth, congestion level, and transmission delay. High-quality paths receive a larger portion of traffic, while congested or degraded paths receive reduced traffic loads. This mechanism improves network throughput, reduces congestion, enhances fault tolerance, and increases overall network reliability. It also enables seamless traffic continuity in case of path failures, as traffic is automatically redistributed across remaining active paths.

B. Fault-Tolerant Network Architecture

- The network is designed with built-in fault tolerance to ensure continuous operation even during failures. Redundant communication paths, alternative routing options, and distributed control mechanisms allow the network to isolate faults and prevent cascading failures.
- The architecture ensures that single-point failures do not disrupt the entire network. By combining redundancy with self-healing and adaptive routing, the system

achieves high resilience and operational stability in complex network environments.

- The project includes a real-time monitoring and analytics framework that continuously observes network performance parameters such as throughput, latency, jitter, packet loss, queue length, and link utilization. This data is used both for performance evaluation and for intelligent decision-making in routing and traffic management.
- Visualization tools and dashboards provide clear insights into network health, traffic flows, and fault conditions, enabling better understanding, debugging, and performance optimization.

C. Performance Evaluation and Optimization Module

The system includes performance evaluation mechanisms that analyze network behavior before and after recovery and optimization. Metrics such as throughput improvement, delay reduction, packet loss reduction, recovery time, and traffic distribution efficiency are measured and compared. These evaluations enable continuous optimization of routing strategies and traffic management policies, ensuring consistent improvement in network performance and reliability.

IV. Required tools and softwares

A. Programming Environment

Python 3.9 or higher Core programming language used for implementing the simulation, routing logic, fault detection, self-healing mechanisms, and multipath traffic management.

Development Environments (IDEs):

VS Code – Code development and debugging

PyCharm – Advanced Python project management

Jupyter Notebook – Data analysis, visualization, and testing

B. Network Modeling and Simulation

- networkx
  Used for creating and managing network graphs, nodes, edges, and routing paths.
- simpy
  Discrete-event simulation framework for modeling network traffic flows, delays, and failures.
- scapy (optional)
  Packet-level simulation and traffic modeling.

C. Data Processing and Computation

- matplotlib
  Graph plotting and performance visualization.
- plotly
  Interactive dashboards and network visualization.
- seaborn (optional)
  Statistical data visualization.

D. Simulation and Networking Platforms

Python-Based Simulation

Custom network simulation environment using Python libraries.

External Simulation Tools (Optional Integration) NS-3

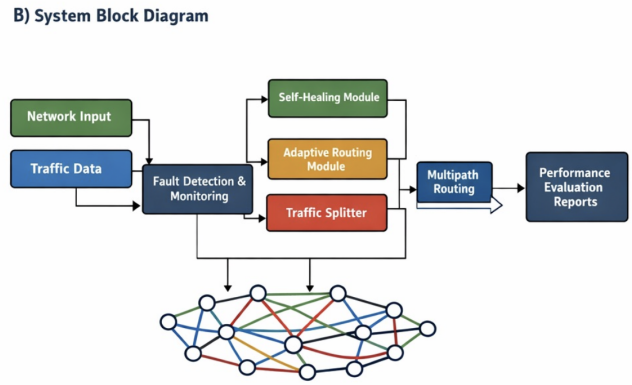

Fig. 1. Block Diagram

Advanced network simulation for realistic network behavior modeling.

Mininet

SDN-based network emulation platform.

E. Development and Project Management Tools

a) Git and GitHub – Version control:

b) Google Drive / OneDrive – Documentation storage:

c) Draw.io / Lucidchart – Architecture and block diagrams:

V. Expected Result

The proposed AI-based Adaptive Multipath Self-Healing Network System is expected to demonstrate significant improvements in network reliability, performance, and intelligence through automated monitoring, intelligent decision-making, and adaptive routing mechanisms. The key expected outcomes are described below:

1. Intelligent Fault Detection and Prediction

The system will continuously monitor network parameters such as latency, packet loss, jitter, throughput, and node availability. Using machine learning models, it will: Detect abnormal traffic patterns and early signs of node/link failure Predict possible network congestion and failures before they occur Classify faults (link failure, node failure, congestion, overload conditions) Expected outcome: Reduced unexpected downtime and faster fault identification compared to traditional static monitoring systems.

2. Self-Healing Network Behavior

Upon detecting faults, the system will automatically trigger the self-healing module, which: Isolates faulty nodes or links Reconfigures routing paths dynamically Redirects traffic without human intervention Expected outcome: Automatic recovery from failures with minimal service disruption.

3. Adaptive Multipath Routing Efficiency

The multipath routing engine will: Maintain multiple alternative paths between source and destination Dynamically select optimal paths based on real-time network conditions

Distribute traffic load intelligently across multiple paths Metrics improved: End-to-end delay Throughput Packet delivery ratio Load balancing efficiency Expected outcome: Higher network performance, better bandwidth utilization, and reduced congestion.

4. AI-Driven Decision Making

The intelligent decision engine will: Learn from historical traffic and fault data Optimize routing strategies over time Improve prediction accuracy with continuous training Expected outcome: Smarter network behavior with increasing performance as data grows (self-learning capability).

5. Real-Time Traffic Optimization

The traffic analyzer and splitter will: Identify congested routes Split traffic dynamically based on link capacity and reliability Prevent bottleneck formation Expected outcome: Smooth traffic flow and stable Quality of Service (QoS).

6. Performance Evaluation and Analytics

The system will generate: Real-time performance dashboards Comparative reports (before vs after optimization) Graphs for latency, throughput, packet loss, and recovery time Expected outcome: Clear measurable proof of system improvement and effectiveness.

7. Scalability and Network Adaptability

The architecture will support: Small networks (LAN, campus networks) Large-scale networks (IoT, smart cities, cloud networks) Dynamic node addition/removal Expected outcome: Highly scalable and future-ready system design.

8. Simulation and Validation Results

Through simulation environments (Mininet / NS3 / Python-based simulators), the system will show: Faster recovery time after failures Improved route stability Lower congestion probability Better overall network resilience Expected outcome: Experimental validation of robustness and reliability.

ACKNOWLEDGMENT

We sincerely thank our respected faculty and the Department of Electronics and Communication , MIT Manipal and our esteemed Professor Dr. Ravilla Dilli for their continuous guidance, support, and encouragement throughout the development of this project. We are grateful for the valuable suggestions and technical insights provided during the course of this work. We also acknowledge the institutional facilities and resources that made this project possible. Lastly, we thank all those who directly or indirectly contributed to the successful completion of this project.

REFERENCES

[1] S. Helmy and C. Kruskal, "Towards network resilience: Models and metrics," Comput. Netw., vol. 52, no. 5, pp. 882–899, Apr. 2008.

[2] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 7th ed. Boston, MA, USA: Pearson, 2017.

[3] [1] S. Nelakuditi, Z. Zhang, and R. P. Tsang, "Adaptive multipath routing," in Proc. IEEE Int. Conf. Commun. (ICC), Anchorage, AK, USA, 2003, pp. 1825–1830.

[4] M. Medard and R. Koetter, "Network resilience: A systematic approach," IEEE Communications Magazine, vol. 42, no. 8, pp. 80–87, Aug. 2004.

[5] IEEE Communications Society, "Resilient and fault-tolerant communication networks," IEEE Network, vol. 24, no. 5, pp. 6–13, Sept. 2010.

[6] X. Wang and X. Liu, "A survey on multipath routing protocols for wireless networks," J. Netw. Comput. Appl., vol. 68, pp. 1–16, Jan. 2016.